

Examen de Desarrollo Javull

Fabrisio
Alfaya Tolentino
22200003

20/06/2026

2. El Pattern de Repositorio como SOAT

1º

Se le considera SSOT porque centraliza el acceso a los datos de la app, además se asegura que exista el único punto de origen para cada pieza de información, la UI y el ViewModel solo concurren datos que solo provengan del repositorio, evitando la inconsistencia cuando hay muchas lecturas de varias fuentes.

2º

Corrutinas y structured concurrency

¿Qué es viewModelScope?

Es como un ciclo de vida que está limitada por CoroutineScope que está relacionada al viewModel, es muy importante en la gestión de la memoria ya que es automática, ya que retiene hilos innecesarios (hilos huérfanos).

Si ocurre eso el viewModel invoca al onCleared(), lo que hace es destruir el scope, lo que hace es cancelar todas las corrutinas que se encuentran activadas en ese momento, liberando la red y evitando los fugas en la memoria.

3º Flujos Reactivos (Cold vs Hot)

Flow → Es de flujo pasivo, no emite datos, ni código hasta que halla un suscriptor activo, para eso es necesario collect(). además se almacena de forma intermedia en la memoria.

Flow (hot) → Es activo y la memoria, mantiene su estado o valor actual, a través de value().

State flow es mejor en el manejo de la interfaz del usuario compose ui porque requiere conocer al estado estable, es decir cuando hay algo extra, etc). además incorpora consecuencias mutuas, lo que evita consecuencias inesperadas.

4º Inyección de Dependencias (DI) Manual

Es instanciar y almacenar los objetos globales (como B.D y Repositorios) como propiedades de la clase personalizada, herediada de `Application Class` en todo por eso de la app)

lazy -> es como "el delegado" de Kotlin -> ayuda en iniciar la iniciación perezosa, es decir solo se crean cuando hay lectura en el código, ayuda disminuir el tiempo inicial, disminuye el consumo de ram y optimiza recursos.

3. II Completar

1º El flujo de datos que emite automáticamente cada vez que una tabla de datos cambia se implementa usando -> `Flow`

2º El operador -> `Dispatchers.Default` permite ejecutar tareas fuera del hilo principal específicamente optimizado para tareas de CPU como parsear masivo de datos.

3º Para que un servicio de primer plano de ubicación se valide en android 10+, se debe declarar en manifiesto tipo -> `location`

4º La función `Container` de la clase `aplicativo` es lo que garantiza que los componentes compartan la misma instancia de repositorio mediante un `cost`.

5º El componente que permite transformar una app basada en `callbacks` antiguos en una función susceptible moderna llamado -> `callback flow`.

6º El archivo de metadatos que describe la estructura de la app al sistema operativo se llama -> `androidManifest.xml`

7º La arquitectura `Android` la capa que actúa como puente estándar entre `framework` y los controles se denomina -> `HAL`

8º El operador de `flow` que permite combinar múltiples fuentes (GPS, Sensores) para emitir todo unificado -> `combine`

4º Desarrollo de código y lógica personalizada

①

Interface UsuarioDao

```
class UsuarioRepository (private val usuarioDao : UsuarioDao) {  
}  
class MobileApplication : Application() {  
    private val Database by lazy {  
        Room.databaseBuilder().build()  
    }  
    private val usuarioDao by lazy {  
    }  
    val UsuarioRepository : UsuarioRepository by lazy {  
        UsuarioRepository (usuarioDao)  
    }  
}
```

② import kotlin.math.max

a) data class Usuario (
 val nombre : String,
 val apellido : String,
 val Edad : Int?
)

fun main () {

b) val miUsuario = Usuario(
 nombre = "Fabricio",
 apellido = "Diego Tolentino",
 Edad = 26
)

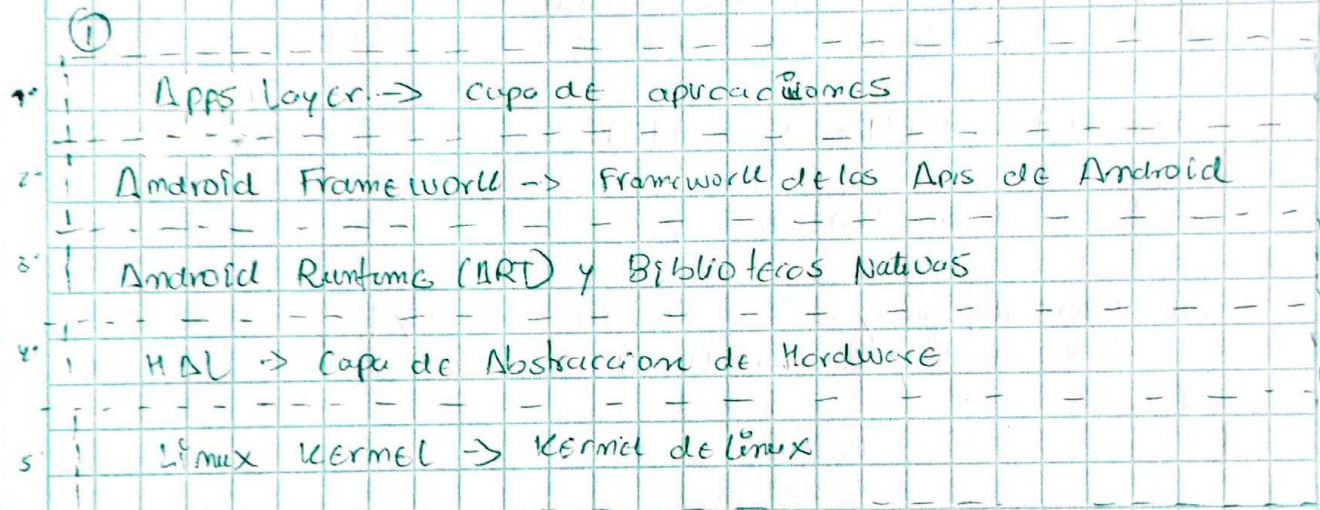
c) val Nombres Limpios = miUsuario.nombre.replace (" ", "")
val Apellidos Limpios = miUsuario.apellido.replace (" ", "")

val Letras Nombres = Nombres Limpios.length
val Letras Apellidos = Apellidos Limpios.length

Vel $\text{punteo} \text{Identidad} = (\text{letras Nombres} + \text{letras Apellidos}) \cdot 10$

d) $\text{println} (" Usuario : [\{ \text{miUsuario, nombre} \} \& \{ \text{miUsuario, apellidos} \}] ,$
tu punteaje identidad es : [$\{ \text{punteo Identidad} \}^{10}$)

5) Arquitectura AOSP



2º

Apps layers $\rightarrow$ Capa superior visible, aloja los apps, nativos, del sistema (apps, contactos etc) y los apps de Terceros (apps publica)

Android Frameworks $\rightarrow$ Bloques modulares (Java/Kotlin) aquí se encuentran los servicios (generales y los Apps, como una app interactúa con el teléfono).

Android Runtime $\rightarrow$ Su rol fundamental es convertir el byte code de las apps en instrucciones de código.

(Art) y los B.N
 $\rightarrow$ componentes $\rightarrow$ compilador AOT, JIT, Garbage collector y los bibliotecas base que se ejecutan en tiempo real

4 Hal → Define las interfaces, pero que framework pueda enviar comandos al hardware físico,

5º Linux Kernel → Núcleo del sistema operativo, es responsable de las comunicaciones de bajo nivel.